

Exercícios OAC

Hierarquia de Memória

Suponha que a memória seja endereçável por *byte* e que as palavras sejam de 32 bits, a menos que especificado o contrário.

5.1 Neste exercício, olhamos para as propriedades de localidade de memória da computação de matrizes. O código a seguir é escrito em C, onde elementos dentro da mesma linha são armazenados contiguamente. Suponha que cada palavra seja um inteiro de 32 bits.

```
for (i=0; i<8; i++)
    for (j=0; j<8000; j++)
        A[i][j]=B[i][0]+A[j][i];
```

5.1.1 Quantos inteiros de 32 bits podem ser armazenados em um bloco de cache de 16 bytes?

5.1.2 Quais referências de variáveis exibem localidade temporal?

5.1.3 Quais referências de variáveis exibem localidade espacial?

A localidade é afetada pela ordem de referência e pelo *layout* dos dados. O mesmo cálculo também pode ser escrito abaixo em Matlab, que difere de C porque armazena elementos de matriz *dentro da mesma coluna* de forma contígua na memória.

```
for I=1:8
    for J=1:8000
        A(I,J)=B(I,0)+A(J,I);
    end
end
```

5.1.4 Quais referências de variáveis exibem localidade temporal?

5.1.5 Quais referências de variáveis exibem localidade espacial?

5.2 Caches são importantes para prover uma hierarquia de memória de alto desempenho para os processadores. Abaixo está uma lista de referências de endereços de memória com palavras de 32 bits. Supor a memória com 256 palavras, de forma que os endereços tem 8 bits.

0x03, 0xb4, 0x2b, 0x02, 0xbf, 0x58, 0xbe, 0x0e, 0xb5, 0x2c, 0xba, 0xfd

5.2.1 Para cada uma dessas referências, escreva o endereço em binário, a *tag* e o índice da *cache*, dado uma *cache* mapeada diretamente com 16 blocos de uma palavra. Liste também se cada referência é um acerto ou erro, supondo que a *cache* esteja inicialmente vazia.

5.2.2 Para cada uma dessas referências, escreva o endereço em binário, a *tag*, o índice e o *offset* da palavra, dado uma *cache* mapeada diretamente com blocos de duas palavras e um tamanho total de oito blocos. Liste também se cada referência é um acerto ou um erro, supondo que a *cache* esteja inicialmente vazia.

5.2.3 Você é solicitado a avaliar configurações de *cache* para as referências fornecidas. Há três *designs* de *cache* mapeados diretamente possíveis, todos com um total de oito palavras de dados. Calcule qual deles apresenta menor taxa de falhas.

- C1 tem blocos de 1 palavra,
- C2 tem blocos de 2 palavras e
- C3 tem blocos de 4 palavras.

5.3 Por convenção, uma *cache* é nomeada de acordo com a quantidade de dados que ela contém (ou seja, uma *cache* de 4 KiB pode conter 4 KiB de dados); no entanto, as *caches* também exigem SRAM para armazenar metadados, como *tags* e bits de validade. Para este exercício, você examinará como a configuração de uma *cache* afeta a quantidade total de SRAM necessária para implementá-la, bem como o desempenho da *cache*. Para todas as partes, suponha que as *caches* sejam mapeadas diretamente, endereçáveis por byte e que os endereços e as palavras sejam de 32 bits.

5.3.1 Calcule o número total de bits necessários para implementar uma cache de 32 KiB com blocos de duas palavras.

5.3.2 Calcule o número total de bits necessários para implementar um cache de 64 KiB com blocos de 16 palavras. Quanto maior é essa *cache* do que a *cache* de 32 KiB descrita no Exercício 5.3.1? (Observe que, ao alterar o tamanho do bloco, dobramos a quantidade de dados sem dobrar o tamanho total da *cache*.)

5.3.3 Explique por que essa *cache* de 64 KiB, apesar de seu tamanho de dados maior, pode fornecer desempenho mais lento do que a primeira *cache*.

5.9 O tamanho do bloco de cache (B) pode afetar tanto a taxa de falhas quanto a latência da falha. Supondo uma máquina com um CPI base de 1 e uma média de 1,35 referências (instrução e dados) por instrução, encontre o tamanho do bloco que minimiza a latência total de falta, dadas as seguintes taxas de falta para vários tamanhos de bloco.

B/Taxa: 8: 4% 16: 3% 32: 2% 64: 1,5% 128: 1%

5.9.1 Qual é o tamanho de bloco ideal para uma latência de falha de $20 \times B$ ciclos?

5.9.2 Qual é o tamanho de bloco ideal para uma latência de falha de $24 + B$ ciclos?

5.9.3 Para latência de falha constante, qual é o tamanho de bloco ideal?

Considerando Average Memory Access Time = AMAT, podemos determinar a latência média do acesso à memória como:

$$\text{AMAT} = \text{Time for a hit} + \text{Miss rate} \times \text{Miss penalty}$$

5.10 Neste exercício, veremos as diferentes maneiras pelas quais a capacidade afeta o desempenho geral. Em geral, o tempo de acesso à *cache* é proporcional à capacidade. Suponha que os acessos à memória principal levem 70 ns e que 36% de todas as instruções acessem a memória de dados. A tabela a seguir mostra dados para *caches* L1 conectados a cada um dos dois processadores, P1 e P2.

	L1 Size	L1 Miss Rate	L1 Hit Time
P1	2 KiB	8.0%	0.66 ns
P2	4 KiB	6.0%	0.90 ns

5.10.1 Supondo que o tempo de acerto L1 determine os tempos de ciclo para P1 e P2, quais são suas respectivas taxas de *clock*?

5.10.2 Qual é o Tempo Médio de Acesso à Memória para P1 e P2 (AMAT - em ciclos)?

5.10.3 Assumindo um CPI base de 1,0 sem nenhuma paralisação de memória, qual é o CPI total para P1 e P2? Qual processador é mais rápido? (Quando dizemos um “CPI base de 1,0”, queremos dizer que as instruções são concluídas em um ciclo, a menos que o acesso à instrução ou o acesso aos dados cause uma falha de cache.)